

scripts/pipeline/phase-05a-changelog-entry.sh — User Guide

Last verified: 2026-08-21 (feature 002-build-test-distribute-pipeline, task T042) **Inheritance:** HelixConstitution §11.4.18 (script documentation mandate); Lava §6.P (Distribution Versioning + Changelog Mandate), §6.J (Anti-Bluff)

Overview

Authors the two documentation artifacts that scripts/firebase-distribute.sh requires to **already exist** before it will distribute anything:

Artifact	Gate it satisfies
an entry in root CHANGELOG.md for <version> (<code>)	firebase-distribute.sh Gate 2 (grep -qE "\$CHANGELOG_PATTERN" CHANGELOG.md)
.lava-ci-evidence/distribute-changelog/<channel>/<version>-<code>.md	firebase-distribute.sh Gate 3 ([[-f "\$SNAPSHOT_FILE"]]) — this file is also what gets shipped as the App Distribution release notes

Per specs/002-build-test-distribute-pipeline/research.md **R-004**, the dependency direction here is inverted from a naive reading of the spec's User Story 3: distribution *needs the CHANGELOG entry to already exist*, not the reverse. So this phase runs **before** phase-05-distribute.sh, while the broader documentation refresh (phase-06-docs.sh) runs **after** it.

What it does NOT do (honest scope)

It satisfies **Gates 2 and 3 only**. It deliberately does not produce anything that would make these pass:

- **Gate 1** (§6.P monotonic versionCode) — that is the §6.Y version-bump commit's job, not a documentation job.
- **Gates 4 + 5** (§6.R/§6.H pepper rotation).
- **Gate 6** (APK-manifest versionCode cross-check).
- **Gate 7** (§6.AK cycle-coverage) — that gate reads the separate cycle-coverage-map-<version>.yaml plus <version>-test-evidence.{md,json} artifacts, *not* this script's output.

Having a documentation-authoring script quietly manufacture artifacts that make an independent safety gate pass would be exactly the §6.J bluff class this project forbids.

Usage

```
scripts/pipeline/phase-05a-changelog-entry.sh <run_id>  
[repo-path] [options]
```

Option	Meaning
--app client api-app both	which artifact(s) to author for (default both)
--notes-file <app>=<path>	use <path>'s contents verbatim as the entry body for <app> instead of auto-deriving from git history (repeatable)
--title <app>=<text>	headline suffix for <app>'s entry (repeatable)
--dry-run	print exactly what <i>would</i> be written to both files, change nothing on disk
--force	overwrite an existing snapshot file. Never duplicates an existing CHANGELOG.md entry

<run_id> must already have a report.json (created by lib/run-report.sh's init_run_report). This script appends one changelog_entry phase entry to that report; it never creates a new run.

Examples:

```
# Review before letting the pipeline write anything
scripts/pipeline/phase-05a-changelog-entry.sh
2026-08-21T19-32-42Z --dry-run

# Author for the client only, with operator-written release notes
scripts/pipeline/phase-05a-changelog-entry.sh
2026-08-21T19-32-42Z \
--app client --notes-file client=/tmp/notes.md --title
client="P0 auth fix"
```

Entry body: where the bullets come from

With no `--notes-file`, the body is generated from the **real commit range** since the previous published version's snapshot last changed (`git log <anchor>..HEAD`, where `<anchor>` is the commit that last touched `.lava-ci-evidence/distribute-changelog/<channel>/<prev-version>-<prev-code>.md`). If no previous snapshot exists to anchor a range, it falls back to `git log -n 20` and **says so explicitly in the entry** rather than implying a real since-last-release range.

Those bullets are a factual commit listing, not a curated list of verified user-visible fixes — the generated text says this in-line. Any user-visible fix **claim** that needs \$6.AK coverage belongs in the cycle-coverage-map, authored separately.

Anti-bluff properties

1. **Drift detection.** The per-app channel names, gradle version files, and Gate-2 regex *templates* are copies of `scripts/firebase-distribute.sh`'s own case `"$SELECTED_APP"` block. At runtime this script greps that file for every literal it copied and **fails loudly** if any is no longer present, instead of silently emitting an entry the real gate would not match.

Each literal is matched **in its assignment context** —

`GRADLE_VERSION_FILE="..."`, `CHANGELOG_CHANNEL="..."`, `CHANGELOG_PATTERN_TMPL='...'` — never as a bare value. That is load-bearing, because the two apps' values are substrings of one another: `firebase-app-distribution` is a *prefix* of `firebase-app-distribution-api-app`, and `app/build.gradle.kts` is a *substring* of `api-app/build.gradle.kts` (both paths also appear in ordinary comment prose in that file). A bare substring grep for the client channel was satisfied by the **api-app** channel's line, so renaming the client channel upstream left the drift check green, this phase wrote the snapshot into the stale channel directory and reported

PASS, and `firebase-distribute.sh` Gate 3 then refused the distribute for a snapshot it could not find. Including the variable name and the closing quote rules out both the longer sibling value and prose. Covered by `tests/pipeline/test_phase_05a_drift_detection.sh`.

2. **The version parse is validated, not just non-empty.** The parse expressions are byte-identical to `firebase-distribute.sh`'s (lines 113-116) and must stay that way — the two have to agree or Gates 2+3 refuse the distribute. But `sed` prints its input **unchanged** on a no-match, and the greps are looser than the seds that follow them (`versionCode\s*=` versus a literal `"= "`). So a valid Kotlin `versionCode=1086` written without spaces passed the grep, failed the sed, and yielded the whole line — non-empty, so the `-z` guard was satisfied and a **no-match parse read as success**: the phase reported PASS while writing `.../firebase-app-distribution/9.9.1-versionCode=9001.md` and a matching CHANGELOG heading. The fix keeps the shared expression and validates its **output**: `versionCode` must be a plain integer (the gate compares it arithmetically), and `versionName` must carry no whitespace or quotes. Real values (1.3.17, 0.2.13, 9.9.2-rc1) all pass. Covered by `tests/pipeline/test_phase_05a_version_parse_sanity.sh`.
3. **Post-write self-verification.** After writing, it re-runs the gate's own two checks — the same `grep -qE "$CHANGELOG_PATTERN"` against the real `CHANGELOG.md`, and the same `[[ -f "$SNAPSHOT_FILE" ]]`. A failure there is reported as this script's own bug (FAIL), never as success.
4. **Idempotent.** An already-present entry for the same version is detected with the gate's own regex and left untouched (no duplicate). An existing snapshot is left alone unless `--force`.

Exit codes

Code	Meaning
0	for every selected app, the CHANGELOG entry and per-version snapshot are present and were re-verified with <code>firebase-distribute.sh</code> 's own Gate 2 + Gate 3 checks; Evidence Record written and anti-bluff-validated (or, under <code>--dry-run</code> , the full would-be output was printed and nothing written)
1	a real failure — drift against <code>firebase-distribute.sh</code> , unparseable version, failed write, failed post-write self-

Code	Meaning
	verification, or an Evidence Record rejected by <code>anti-bluff-validate.sh</code> . Recorded as FAIL in <code>report.json</code> , never fabricated as success
2	usage/precondition error — missing <code>run_id</code> , absent <code>report.json</code> , bad <code>--app</code> value, unreadable <code>--notes-file</code>

Per-app raw evidence (2026-08-22)

A run with `--app` both writes one Evidence Record per app. Each record cites its **own** raw output file:

```
.lava-ci-evidence/pipeline-runs/<run-id>/phase-05a/hermetic-
script/raw/
  changelog-entry-client.log      <- raw_output_ref of the client
record
  changelog-entry-api-app.log    <- raw_output_ref of the api-app
record
  changelog-entry-combined.log   <- the whole run's narrative,
cited by neither
```

Before this change both records carried the same `raw_output_ref` (`raw/changelog-entry-combined.log`), which made neither claim independently falsifiable: if one app's CHANGELOG entry were wrong, the file its record points at also holds the other app's successful output, and no line in it is attributable to one record rather than the other — a FAILED app and a PASSED app would have cited byte-identical evidence.

Each per-app file is real captured output (every `_log/_fail` line for that app is tee'd into it as it is produced, not re-derived afterwards) and opens with a provenance header naming the app, the repo, the run id, and the combined log. `tests/pipeline/test_phase_05a_per_app_raw_evidence.sh` asserts the two refs differ, that each resolves to a non-empty file, that each file contains its own app's release label, and that neither contains the other's.

Maintenance

When this script is modified, update this document in the same commit (§11.4.18 / CM-SCRIPT-DOCS-SYNC convention). If `scripts/firebase-distribute.sh`'s per-app case block changes its channel names or Gate-2 pattern templates, this script's `_resolve_app_config` **must** be re-synced — its own drift check will fail the pipeline until you do.

Cross-references

- `scripts/pipeline/phase-05a-changelog-entry.sh` — the script itself
- `scripts/firebase-distribute.sh` — the consumer whose Gates 2+3 this satisfies
- `scripts/pipeline/phase-06-docs.sh` — the *post*-distribution documentation phase
- `specs/002-build-test-distribute-pipeline/research.md` — R-004 (ordering rationale)
- `specs/002-build-test-distribute-pipeline/contracts/cli-contract.md` — phase-script contract table
- `Lava CLAUDE.md` §6.P (Distribution Versioning + Changelog Mandate), §6.AK (cycle coverage)